

Noise Model Simulator - Breast Cancer (2 Klassen, 2 Features)

Modelle: Quantum Kernel SVM + VQC (RealAmplitudes)

Datensatz: Breast Cancer (2 Klassen, 2 Features)

Backend: AerSimulator + IBM Noise Model (ibm_marrakesh)

Referenz: 04_breast_cancer_fair.ipynb (Ideal Simulator)

Simuliert Hardware-Rauschen lokal - kein QPU-Kontingent verbraucht.

Zeigt den Effekt von Dekohärenz und Gate-Fehlern auf die Klassifikationsqualität.

0. Imports

```
In [1]: import sys, os
sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.svm import SVC

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.algorithms import VQC
from qiskit_machine_learning.optimizers import SPSA
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel
from qiskit_aer.primitives import Sampler as AerSampler
from qiskit_ibm_runtime import QiskitRuntimeService

from utils import save_result

print("Imports OK")
```

Imports OK

1. Noise Model laden

```
In [2]: # IBM Account verbinden
service = QiskitRuntimeService()

# Noise Model von ibm_marrakesh laden
backend = service.backend("ibm_marrakesh")
noise_model = NoiseModel.from_backend(backend)

# AerSimulator mit Noise Model
noisy_simulator = AerSimulator(noise_model=noise_model)

print(f"Noise Model geladen von: {backend.name}")
print(f"Basis-Gates: {noise_model.basis_gates}")
```

```
qiskit_runtime_service.__init__:WARNING:2026-05-01 08:59:12,387: Instance was not set at service instantiation. Free and trial plan instances will be prioritized. Based on the following filters: (tags: None, region: us-east, eu-de), and available plans: (open), the available account instances are: open-instance. If you need a specific instance set it explicitly either by using a saved account with a saved default instance or passing it in directly to QiskitRuntimeService().
qiskit_runtime_service.backends:WARNING:2026-05-01 08:59:12,390: Using instance: open-instance, plan: open
Noise Model geladen von: ibm_marrakesh
Basis-Gates: ['cz', 'delay', 'id', 'if_else', 'measure', 'reset', 'rz', 'sx', 'x']
```

2. Datensatz - identische Pipeline wie 04_breast_cancer_fair

```
In [3]: bc = load_breast_cancer()
X = bc.data
y = bc.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

pca = PCA(n_components=2, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train_pca)
X_test_sc = scaler.transform(X_test_pca)

print(f"Erklärte Varianz: {pca.explained_variance_ratio_.sum():.2%}")
print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")
```

```
Erklärte Varianz: 99.93%
Train: (398, 2) | Test: (171, 2)
```

3. Quantum Kernel SVM (Noise Model)

```
In [5]: feature_map_qk = zz_feature_map(feature_dimension=2, reps=2)

# FidelityQuantumKernel mit Noisy Simulator
kernel = FidelityQuantumKernel(feature_map=feature_map_qk)

# Noisy Sampler
from qiskit_aer.primitives import SamplerV2 as NoisySampler
noisy_sampler = NoisySampler.from_backend(noisy_simulator)
kernel = FidelityQuantumKernel(feature_map=feature_map_qk, fidelity=None)

start = time.time()
svc_q = SVC(kernel=kernel.evaluate)
svc_q.fit(X_train_sc, y_train)
train_time_qk = round(time.time() - start, 4)

start = time.time()
y_pred_qk = svc_q.predict(X_test_sc)
infer_time_qk = round(time.time() - start, 4)

acc_qk = accuracy_score(y_test, y_pred_qk)
f1_qk = f1_score(y_test, y_pred_qk, average="binary")

print(f"Accuracy: {acc_qk:.4f} | F1: {f1_qk:.4f}")
print(f"Training: {train_time_qk}s | Inferenz: {infer_time_qk}s")
print()
print(classification_report(y_test, y_pred_qk, target_names=bc.target_names))
```

```
Accuracy: 0.8772 | F1: 0.9041
Training: 399.4972s | Inferenz: 345.1652s
```

	precision	recall	f1-score	support
malignant	0.86	0.80	0.83	64
benign	0.88	0.93	0.90	107
accuracy			0.88	171
macro avg	0.87	0.86	0.87	171
weighted avg	0.88	0.88	0.88	171

4. VQC (Noise Model)

```
In [6]: feature_map_vqc = zz_feature_map(feature_dimension=2, reps=1)
ansatz = real_amplitudes(num_qubits=2, reps=2)

# VQC mit Noisy AerSampler
noisy_sampler_vqc = NoisySampler.from_backend(noisy_simulator)

vqc = VQC(
    feature_map=feature_map_vqc,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=150),
    sampler=noisy_sampler_vqc,
)
```

```

print("Starte VQC Training (Noise Model)...")
start = time.time()
vqc.fit(X_train_sc, y_train)
train_time_vqc = round(time.time() - start, 4)

start = time.time()
y_pred_vqc = vqc.predict(X_test_sc)
infer_time_vqc = round(time.time() - start, 4)

acc_vqc = accuracy_score(y_test, y_pred_vqc)
f1_vqc = f1_score(y_test, y_pred_vqc, average="binary")

print(f"Accuracy: {acc_vqc:.4f} | F1: {f1_vqc:.4f}")
print(f"Training: {train_time_vqc}s | Inferenz: {infer_time_vqc}s")
print()
print(classification_report(y_test, y_pred_vqc, target_names=bc.target_names))

```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Starte VQC Training (Noise Model)...

Accuracy: 0.8421 | F1: 0.8831

Training: 4600.4033s | Inferenz: 9.5284s

	precision	recall	f1-score	support
malignant	0.89	0.66	0.76	64
benign	0.82	0.95	0.88	107
accuracy			0.84	171
macro avg	0.86	0.80	0.82	171
weighted avg	0.85	0.84	0.84	171

5. Ergebnisse speichern

```

In [7]: DATENSATZ = "Breast Cancer (2 Klassen, 2 Features, noise)"

save_result("Quantum Kernel SVM (Noise)", DATENSATZ, "AerSimulator+Noise",
save_result("VQC (RealAmplitudes, Noise)", DATENSATZ, "AerSimulator+Noise",

print("Gespeichert.")

```

Backend	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s	Feature Map	Rep
Quantum Kernel SVM Iris (3 Klassen, 2 Features, fair)						
AerSimulator 2.0	0.6222	0.616		28.2243	23,517	zz_feature_map
VQC (RealAmplitudes) Iris (3 Klassen, 2 Features, fair)						
AerSimulator 1.0	0.5333	0.4222		261.5463	0.3237	zz_feature_map
Klassischer SVM (RBF) Iris (3 Klassen, 2 Features, fair)						
lokal	0.9556	0.9556	0.0114	0.0012	NaN	NaN
MLP Iris (3 Klassen, 2 Features, fair)						
lokal	0.9333	0.9327	0.3173	0.0012	(64, 32)	NaN
Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 2.0	0.8772	0.9041		408.9585	352.8897	zz_feature_map
VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 1.0	0.8713	0.9027		1004.3609	1.2259	zz_feature_map
Klassischer SVM (RBF) Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9181	0.9381	0.0076	0.0018	NaN	NaN
MLP Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9240	0.9412	0.9143	0.001	(64, 32)	NaN
Quantum Kernel SVM (Noise) Iris (3 Klassen, 2 Features, noise)						
AerSimulator+Noise 2.0	0.6222	0.616		28.2971	24.354	zz_feature_map
VQC (RealAmplitudes, Noise) Iris (3 Klassen, 2 Features, noise)						
AerSimulator+Noise 1.0	0.5778	0.4617		3366.2989	8.4769	zz_feature_map
Quantum Kernel SVM (Noise) Breast Cancer (2 Klassen, 2 Features, noise)						
AerSimulator+Noise 2.0	0.8772	0.9041		399.4972	345.1652	zz_feature_map

```

Simulator+Noise      0.6222  0.616          28.2971          24.354 zz_feature_
map      2.0
VQC (RealAmplitudes, Noise)      Iris (3 Klassen, 2 Features, noise) Aer
Simulator+Noise      0.5778  0.4617          3366.2989          8.4769 zz_feature_
map      1.0
Quantum Kernel SVM (Noise) Breast Cancer (2 Klassen, 2 Features, noise) Aer
Simulator+Noise      0.8772  0.9041          399.4972          345.1652 zz_feature_
map      2.0
VQC (RealAmplitudes, Noise) Breast Cancer (2 Klassen, 2 Features, noise) Aer
Simulator+Noise      0.8421  0.8831          4600.4033          9.5284 zz_feature_
map      1.0
Gespeichert.

```

6. Vergleich: Ideal vs. Noise Model

```

In [3]: df = pd.read_csv("Ergebnisse/ergebnisse.csv")

df_ideal = df[df["Datensatz"] == "Breast Cancer (2 Klassen, 2 Features, fair
df_ideal.columns = ["Modell", "Accuracy (Ideal)", "F1 (Ideal)"]

df_noise = df[df["Datensatz"] == "Breast Cancer (2 Klassen, 2 Features, nois
df_noise.columns = ["Modell", "Accuracy (Noise)", "F1 (Noise)"]

# Modellnamen vereinheitlichen für Merge
df_noise["Modell"] = df_noise["Modell"].str.replace(" (Noise)", "", regex=False)

df_cmp = pd.merge(df_ideal, df_noise, on="Modell", how="outer")
df_cmp["Δ Accuracy"] = (df_cmp["Accuracy (Noise)"] - df_cmp["Accuracy (Ideal)"])

print(df_cmp.to_string(index=False))

```

	Modell	Accuracy (Ideal)	F1 (Ideal)	Accuracy (Noise)	F1 (Noise)	Δ Accuracy
1 (Noise)	Klassischer SVM (RBF)	0.9181	0.9381	NaN	NaN	NaN
NaN	MLP	0.9240	0.9412	NaN	NaN	NaN
NaN	Quantum Kernel SVM	0.8772	0.9041	0.8772	0.9041	0.0
NaN	VQC (RealAmplitudes)	0.8713	0.9027	NaN	NaN	NaN
NaN	VQC (RealAmplitudes, Noise)	NaN	NaN	0.8421	0.8831	NaN